Build Plan Addendum

Local AI Coding Assistant — Review Notes, Environment Strategy, and Working Agreement for Claude Code

This document supplements the original **Final Build Plan — Local AI Coding Assistant**. It does not replace the original plan; it adds review feedback, a concrete development environment strategy, and a division of labor between the human developer and Claude Code (the AI pair-programming tool used to build this project). This file is meant to be given to Claude Code, alongside the project's **CLAUDE.md** file, as onboarding context at the start of the build.

1. Project Summary (Unchanged)

The goal remains: a fully local AI coding agent/assistant, built primarily in Python, capable of working across languages (Python, HTML, CSS, JS, etc.), using an open-source local LLM (e.g. Qwen Coder or DeepSeek Coder) served via Ollama/llama.cpp/vLLM. No third-party APIs are used for the agent's core reasoning. The system architecture (Agent Controller, Planning/ReAct loop, Repository Understanding, Tool System, Safe Code Modification, State Management, Testing Loop, Context Management, Team Memory) stays as defined in the original plan.

2. Review Feedback on the Original Plan

2.1 Scope and Sequencing

The plan is architecturally sound and correctly sequenced (LLM setup → repo understanding → tools → agent loop → safety/recovery → context management → team features). However, each phase listed is individually a substantial sub-project — comparable in scope to components of tools like Aider, Cline, or Continue.dev. Treat Phases 0–3 (a working, repo-aware agent that can safely make one small edit) as the first real milestone, not a quick warm-up.

2.2 Local Model Reliability for Agentic Tool Use

This is the single biggest practical risk in the plan and deserves explicit engineering effort. Small/quantized local models (7B–14B class) are noticeably weaker than premium hosted models at multi-step planning and reliably formatted tool calls. Expect, and design for:

- Malformed or partial JSON in tool-call output
- False-positive "task complete" signals
- Degrading plan quality over long ReAct loops as context grows

> ■ *Recommendation: build defensive parsing, schema validation, and retry/repair logic around every tool call from the start. In most local-model agent projects this validation layer ends up being roughly a third of the real engineering effort — it is not a minor add-on to Section 7.*

2.3 Multi-Language Support Requires a Concrete Parsing Strategy

The AST-Based Code Indexer (Section 5 of the original plan) needs a parser per language. Python's built-in `ast` module covers Python only. For HTML/CSS/JS and future languages, commit now to **tree-sitter** as the parsing layer — it has grammars for essentially all mainstream languages under one consistent API. Deciding this early changes how the indexer is architected from the first line of code.

2.4 Context Budgeting Should Start Earlier

Context Management is Phase 10 in the original sequencing, but even basic repository scanning in Phase 1 will exceed usable context on any non-trivial project. Pull a minimal context-budgeting mechanism (simple truncation/prioritization) into Phase 1, and treat the full compression/summarization system as the later, more sophisticated version.

3. Architectural Consequence of Running the Model Locally

3.1 Decoupling Where You Code from Where the Model Runs

Running the local model through Ollama (or an OpenAI-compatible llama.cpp server) exposes it as a local HTTP endpoint. The Python application communicates with that endpoint through the Model Interface layer already defined in the original plan — it does not need to know or care where the model process physically lives.

```
Python Agent
  |
Model Interface   (HTTP client, model-agnostic)
  |
Ollama / llama.cpp server   (can run on a different machine than the IDE)
  |
Qwen / DeepSeek / other local model
```

Practical consequence: the coding machine (running VS Code + Claude Code) does not need a GPU at all. Only the machine hosting inference does. This makes the Model Interface abstraction non-optional — build it even in the earliest prototype.

4. Working with Claude Code: Division of Labor

Claude Code is used to build the harness — the conventional Python application around the model — and can also produce a first draft of the model-facing prompts and control logic. The line is not "who writes the first version" but "who owns iteration once the model is actually running." Tuning against real, observed model behavior is empirical and model-specific, and should move to the human developer once testing begins.

4.1 Delegate to Claude Code (including first drafts)

- Tool classes (file read/write/edit, code search, command execution, git integration) with clear method signatures and validation
- The Model Interface abstraction and its Ollama adapter
- The AST/tree-sitter based repository indexer
- The patch/search-replace engine and its validation logic

- State snapshot and rollback system
- Test scaffolding, CI-style verification hooks
- A first-draft system prompt and ReAct loop structure, based on established agentic prompting patterns — expect to rewrite this once real model output is observed

4.2 Keep Hands-On: Iteration Against the Real Model

- Refining the system prompt and tool-call format once real Ollama output is observed
- Adjusting the ReAct loop's prompt structure (what an "observation" looks like after a tool runs, how much context to include)
- Retry/repair behavior when the model emits malformed tool-call output — thresholds and strategy come from watching real failures
- Context compression tuning — how aggressively to summarize before the model's effective attention degrades

> ■ *Rule of thumb: Claude Code can write v1 of anything, including prompts. Once you're testing against the actual local model and seeing real failures, iteration on prompts and control logic moves to the human developer — Claude Code isn't watching the model run and can't observe what's actually going wrong.*

5. Recommended First Milestone: A Minimal Vertical Slice

Before fully building out Phase 1 (Repository Understanding) in isolation, build a thin end-to-end slice: Phase 0 (talk to the local model) plus a trivial single-tool version of Phase 2 (e.g. a "read file" tool) wired through a minimal planner.

- Produces a working, demoable loop within days rather than after every subsystem is finished separately
- Surfaces the model's real tool-calling reliability early — this materially affects downstream design decisions
- Gives a skeleton to extend, rather than integrating fully-built subsystems for the first time at the end

6. Handoff-Friendly Repository Practices

Since this project may later be transferred to another developer, these practices should be in place from the first commit, not retrofitted later:

- **Git from commit #1** — develop in a proper repository, not a loose folder converted later.
- **Config-driven, not hardcoded** — model name, endpoint URL, and paths live in config.yaml / .env.
- **.gitignore model weights and virtual environments** — document how to pull the model instead of committing it.
- **requirements.txt / pyproject.toml** pinned to tested versions.
- **README with a 5-minute quickstart** — install Ollama, pull model, install dependencies, run.
- **CLAUDE.md at project root** — persistent project context for Claude Code, and a legible reference for a human successor.

- A Dockerfile is worth adding later, once the architecture stabilizes — not before, to avoid friction while design is still moving.

7. Suggested Folder Structure

```
local-ai-coding-agent/
  agent_controller/      # orchestration, planning, ReAct loop
  model_interface/       # all LLM communication goes through here
  tools/                 # file, search, execution, git tools
  repo_index/            # tree-sitter scanning + retrieval
  state/                 # snapshots, rollback, change tracking
  tests/
  config.yaml            # model name, endpoint, paths (never hardcoded)
  CLAUDE.md               # persistent project context for Claude Code
  README.md
  requirements.txt
  .gitignore
  main.py
```

8. How to Use This Document

Provide this addendum together with the original Final Build Plan and the project's CLAUDE.md file as the first prompt to Claude Code when initializing the repository. A reasonable first instruction is to scaffold the folder structure in Section 7, create the Model Interface abstraction and Ollama adapter, set up config.yaml, and draft a first version of the system prompt / ReAct loop structure — understanding that the prompt draft will be iterated on by the human developer once it is tested against the real running model, per Section 4.